Heart Disease Prediction MLOps

End-to-end machine learning lifecycle

MTech MLOps Assignment | Prepared 18 October 2026

Submission details

Name: Nandini Sharma | Roll ID: 2025AE05259

Course: MTech MLOps | Assignment 1

Executive summary

This project builds a reproducible heart-disease classifier, tracks experiments in MLflow, and serves the fitted pipeline through FastAPI. Docker packages the API and model; Kubernetes manifests define a two-replica Deployment and LoadBalancer Service. Prometheus and Grafana provide metrics and dashboards, while GitHub Actions runs linting, training, tests, image building, and API smoke checks.

Random Forest was selected by mean five-fold CV ROC-AUC (0.9030 versus 0.9025 for Logistic Regression). Logistic Regression scored higher on holdout ROC-AUC and accuracy; both results are reported for transparency.

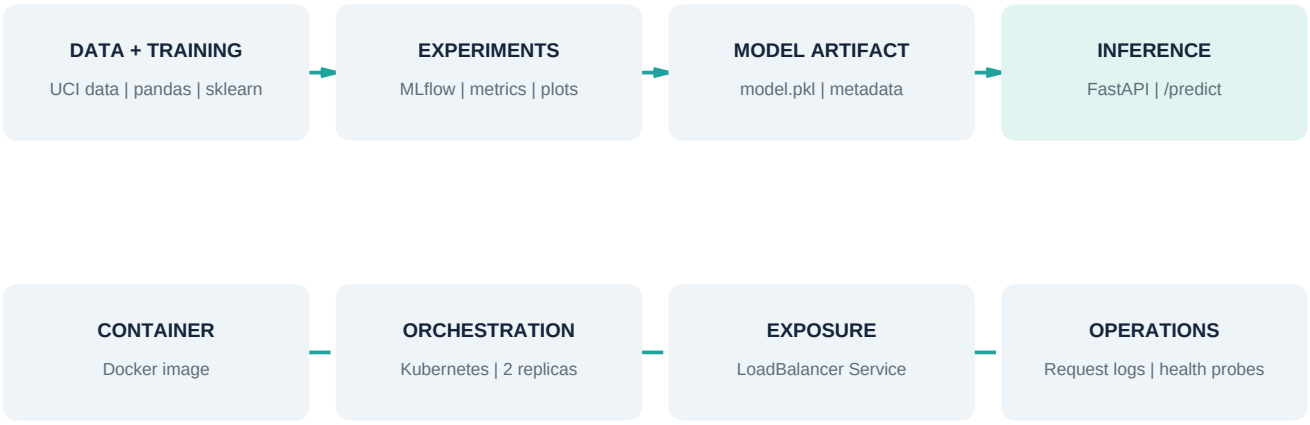
RECORDS	INPUT FIELDS	SELECTED CV AUC	HOLDOUT RECALL
303	13	0.9030	0.9286
Cleveland subset	clinical attributes	Random Forest	Random Forest

Repository: github.com/NandiniSharma05022002/heart_disease_assignment1
Companion demo file: mlops_assignment1_demo_vid.mp4

System architecture

Data, model, serving, deployment, and observability

The architecture keeps training and serving reproducible. A cleaned tabular dataset feeds a shared scikit-learn preprocessing pipeline and two candidate estimators. The final fitted pipeline is stored as an artifact and loaded by the API process. Docker packages the API with that artifact; Kubernetes runs the container and provides service discovery.



Lifecycle flow

- **Prepare:** download the UCI archive, convert the target to binary, and preserve missing feature values for pipeline imputation.
- **Develop:** compare Logistic Regression and Random Forest with stratified cross-validation and a held-out test split.
- **Track and package:** log parameters, metrics, plots, and fitted models to MLflow; save the selected complete pipeline for API inference.
- **Validate and run:** CI executes lint, training, tests, Docker build, and smoke checks. Minikube deployment is applied from the checked-in Kustomize manifests.
- **Observe:** request logs and Prometheus counters/histograms are exposed by the API and visualized in Grafana.

Components in the repository

Area	Files / service
Training	src/heart_disease_mlops/train.py, pipeline.py
Serving	src/heart_disease_mlops/api.py, Dockerfile
Deployment	k8s/deployment.yaml, service.yaml, kustomization.yaml
Monitoring	k8s/prometheus*.yaml, k8s/grafana.yaml, grafana/
Automation	tests/, .github/workflows/ci.yml

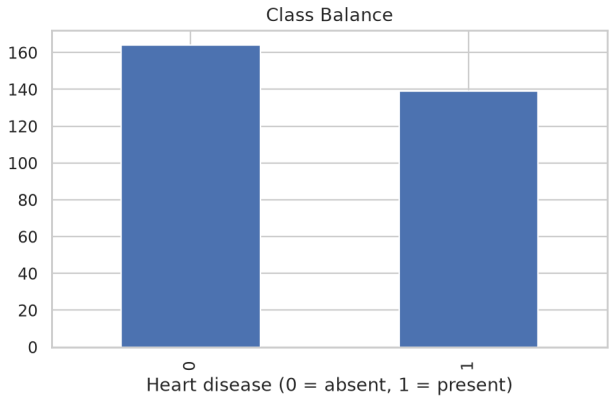
Dataset and preparation

Cleveland subset from the UCI Heart Disease dataset

The processed file in the repository contains 303 rows and 13 predictor columns. UCI describes the Cleveland data as a classification dataset with 303 instances and a goal value from 0 to 4; this project maps values 1 through 4 to disease-present (1) and 0 to disease-absent (0). The source includes missing values and the project preserves them for model-pipeline imputation. [1]

Observed dataset profile

Measure	Observed value
Records	303
Predictor features	13
Target classes	0 = absent; 1 = present
Class 0 count	164 (54.1%)
Class 1 count	139 (45.9%)
Missing ca values	4
Missing thal values	2
Missing target values after cleaning	0



Preparation decisions

- `?` is parsed as missing for numeric columns.
- Rows with an absent target are removed before binary mapping.
- Feature missingness is not filled globally; the estimators impute it within each training fold.
- The class distribution is fairly balanced, so class-sensitive metrics are reported alongside accuracy.

Figure 1. Target balance in the processed Cleveland subset. Counts are read from the saved processed CSV.

Reproducible data commands

From the repository root, the download script creates `data/raw/`; the preparation script creates `data/processed/` and writes `heart_disease.csv`. The processed file is used by the training job.

```
python scripts/download_data.py
python scripts/prepare_data.py
python scripts/eda.py
```

Source: [1] Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1989). Heart Disease [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C52P4X. https://archive.ics.uci.edu/dataset/45/heart%2Bdisease

Exploratory data analysis

Distribution, class balance, and pairwise correlation views

EDA was generated from the cleaned Cleveland subset. The visualizations are used to understand the feature ranges, category distribution, imbalance, and broad pairwise associations before training. They are descriptive checks: they do not imply causal relationships or clinical effects.

Feature distributions

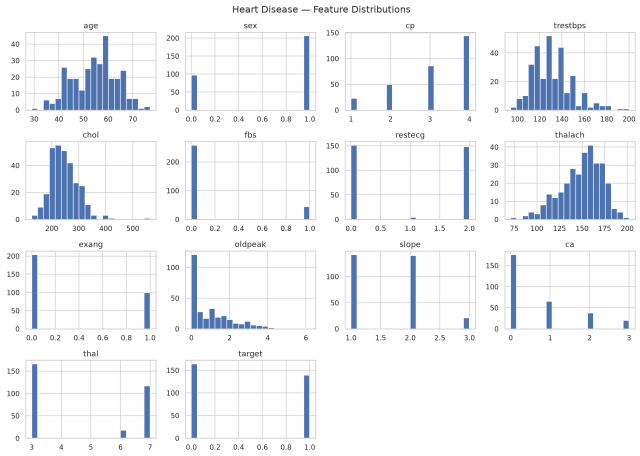


Figure 2. Feature distribution plots for the 13 input variables. Numeric values span different scales, supporting standardization for the numeric branch; categorical codes are treated categorically in the model pipeline.

Correlation heatmap

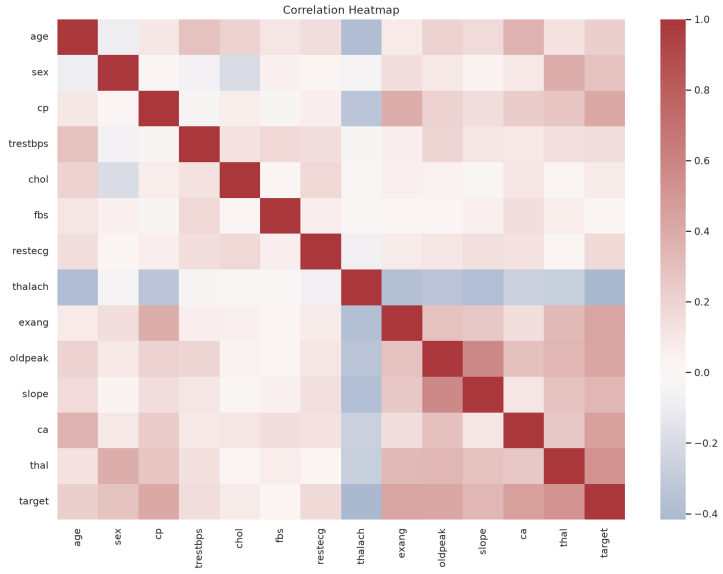


Figure 3. Pearson correlation heatmap across numeric encodings. Some exercise-related variables show visible association with the binary target; encoded categories make the map exploratory rather than a formal measure of categorical dependence.

Interpretation limits

The dataset is small, and correlations between integer-coded categories may be misleading. Model conclusions are therefore based on stratified validation metrics rather than visual patterns alone. Missing values in `ca` and `thal` are handled inside the estimator pipeline.

Feature processing and model design

Preprocessing is fitted inside each cross-validation fold

The feature transformer separates five numeric variables from eight categorical codes. Both candidate models use the same transformation sequence so model performance is compared on a consistent input representation.

Pipeline branches

Feature group	Columns	Processing
Numeric (5)	age, trestbps, chol, thalach, oldpeak	Median imputation -> StandardScaler
Categorical (8)	sex, cp, fbs, restecg, exang, slope, ca, thal	Most-frequent imputation -> one-hot encoding (ignore unseen categories)

Candidate classifiers and search spaces

Candidate	Base configuration	Tuned parameter
Logistic Regression	C=1.0; max_iter=2000; random_state=42	C in {0.1, 1.0, 10.0}
Random Forest	300 trees; class_weight=balanced; random_state=42	min_samples_leaf in {1, 2, 4}

Evaluation protocol

- One stratified 80/20 train-test split uses seed 42.
- Five-fold StratifiedKFold cross-validation runs on the training split.
- GridSearchCV selects parameters with ROC-AUC and refits the best candidate.
- Holdout metrics are computed once for final comparison; the holdout set is not used to tune parameters.
- The serving decision threshold is 0.5; the API returns the positive-class probability as well as the predicted label.

Why pipeline-level preprocessing matters

Imputation, scaling, and one-hot category discovery are learned from each training fold and reused unchanged at prediction time. This reduces leakage risk and keeps model training and API feature treatment aligned.

Model evaluation

Saved cross-validation and holdout metrics

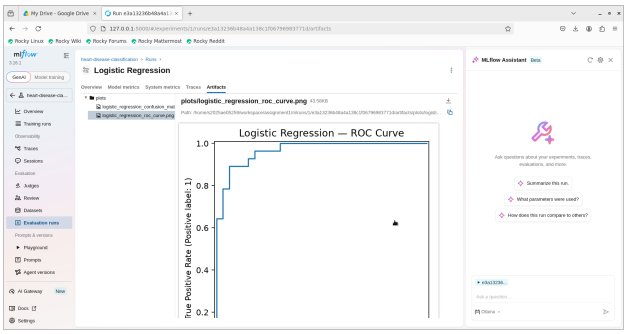
The final comparison artifact records the best parameter setting for each candidate. The model-selection rule is mean five-fold CV ROC-AUC. The highest value belongs to Random Forest by a small margin, while Logistic Regression performs better on this particular holdout set for accuracy, precision, and ROC-AUC.

Metrics from artifacts/model_comparison.csv

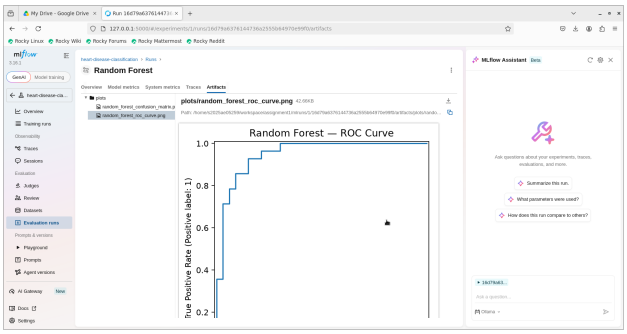
Model	CV Acc.	CV Prec.	CV Recall	CV AUC	Test Acc.	Test Prec.	Test Recall	Test AUC
Logistic Regression	0.8471	0.8768	0.7834	0.9025	0.8852	0.8387	0.9286	0.9665
Random Forest	0.7932	0.7749	0.7826	0.9030	0.8689	0.8125	0.9286	0.9491

Bold-selection rule: Random Forest CV ROC-AUC = 0.9030. Holdout set size is 61 cases (20% of 303 rounded by the split implementation).

ROC curves

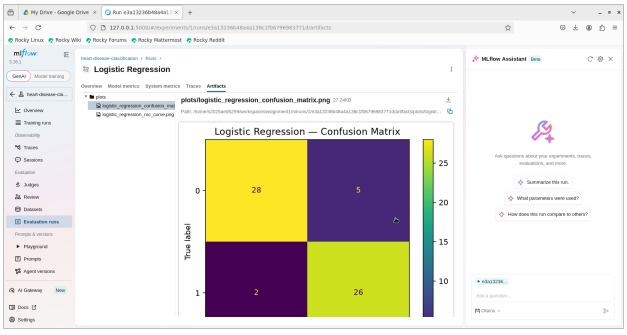


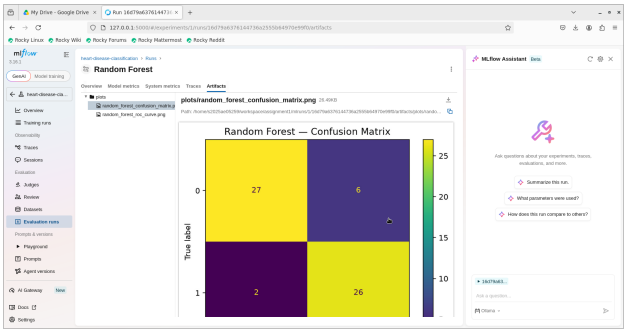
Logistic Regression | holdout AUC 0.9665



Random Forest | holdout AUC 0.9491

Confusion matrices





Both models have holdout recall 0.9286. On this split they identify the same fraction of positive cases, while Logistic Regression has higher precision and fewer overall errors. Model choice should still follow the declared CV criterion and be revisited with external validation.

Experiment tracking and artifacts

MLflow records parameters, metrics, models, and diagnostic plots

Training logs two completed runs under the `heart-disease-classification` experiment using a SQLite tracking backend. The captured experiment page shows both model runs in Finished state. Run-level screenshots show the eight CV/holdout metrics and run IDs.

Experiment overview

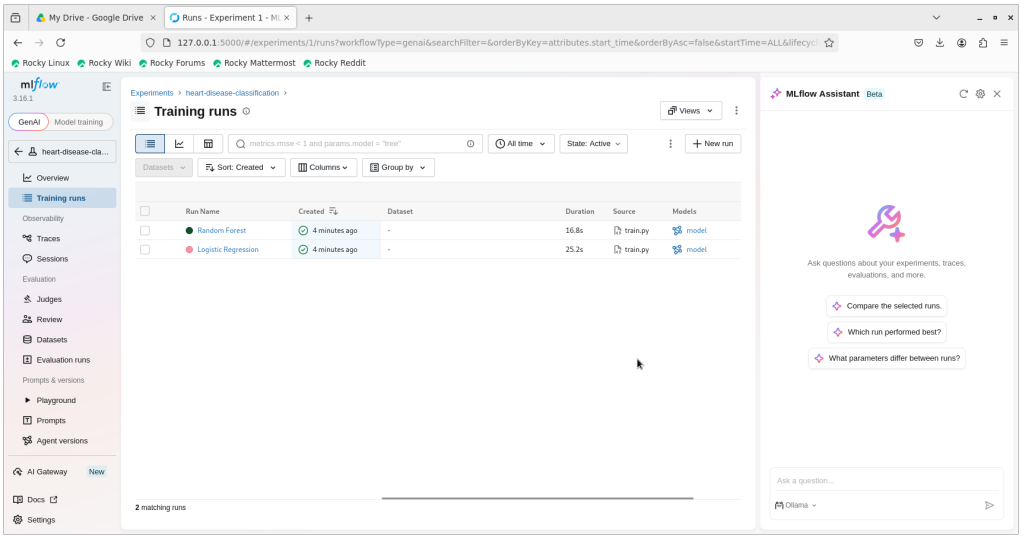


Figure 4. MLflow experiment view with completed Logistic Regression and Random Forest runs.

Run IDs and selection

Run	Run ID	Best search parameter	CV ROC-AUC
Logistic Regression	dbdc672f057e4d0f933c7b1d3db3111e	classifier__C = 1.0	0.902493
Random Forest	8201396213bb4743b0b5b8af12f9b1b2	classifier__min_samples_leaf = 4	0.902985

Tracked artifacts and reproducibility

- MLflow parameters include seed, split size, fold count, candidate name, estimator configuration, and tuned parameter values.
- Metrics include mean CV accuracy, precision, recall, ROC-AUC, and holdout accuracy, precision, recall, and ROC-AUC.
- Each run logs a confusion matrix, ROC curve, and fitted sklearn model.
- The project also writes `artifacts/model.pkl`, `model_metadata.json`, and `model_comparison.csv` for reproducible serving and reporting.

The fitted pickle is trusted-source only. Do not load model pickle files from unknown origins.

Automated test and lint evidence

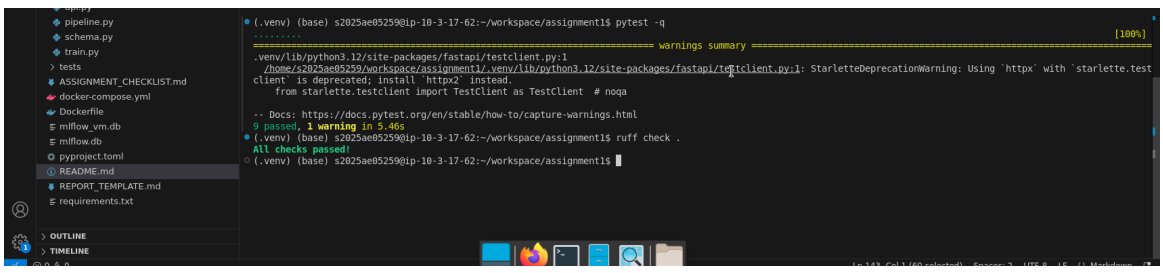


Figure 5. Submitted terminal capture shows 9 Pytest cases passed and Ruff checks passed; one FastAPI deprecation warning is visible.

Inference API and Docker

FastAPI validates JSON input and serves the fitted pipeline

The API loads the saved model once per process and exposes `/health`, `/predict`, `/metrics`, and `/docs`. Pydantic validates the 13 feature fields and their value ranges/categories. `/predict` returns a binary prediction, a label, confidence, and positive-class probability.

API contract

POST /predict request body

```
{ "age": 63, "sex": 1, "cp": 3, "trestbps": 145, "chol": 233,
"fbs": 1, "restecg": 0, "thalach": 150, "exang": 0,
"oldpeak": 2.3, "slope": 2, "ca": 0, "thal": 3 }
```

- **GET /health:** loads/checks the model and returns service status.
- **GET /metrics:** Prometheus text exposition for API and model metrics.
- **GET /docs:** interactive OpenAPI page for endpoint exploration.

Docker evidence and runtime

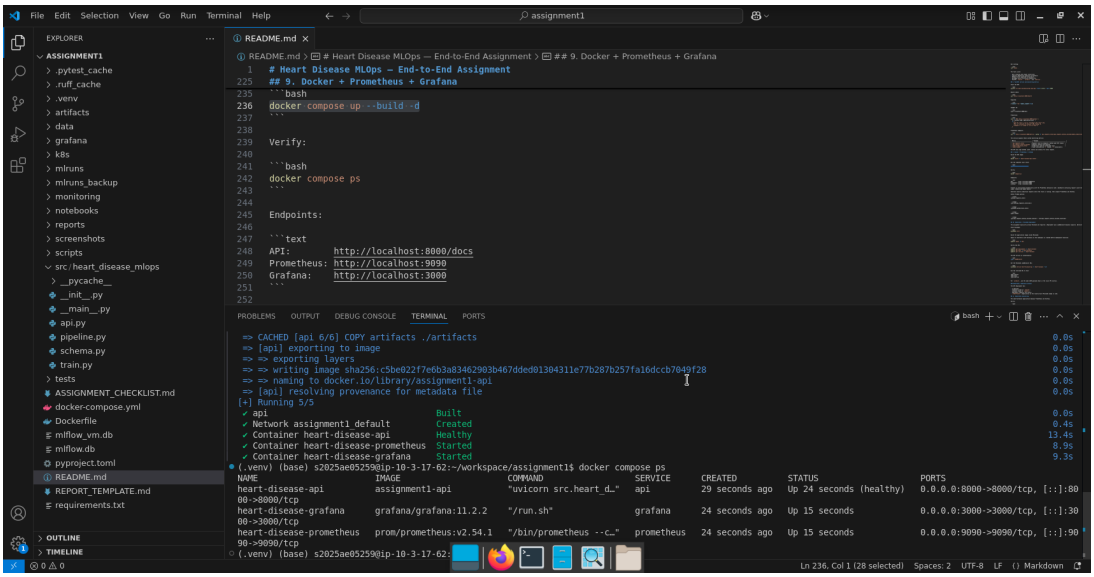


Figure 6. The terminal capture shows Docker Compose services created and the API container reported healthy, with Prometheus and Grafana started.

The Dockerfile uses Python 3.11 slim, installs requirements, copies the application and model artifact, exposes port 8000, and starts Uvicorn. Docker Compose also mounts the Prometheus scrape configuration and Grafana provisioning files. A health check calls `/health`.

Example: `docker build -t heart-disease-api:local .` then `docker compose up --build -d`. The supplied screenshot is local Docker evidence; it does not by itself prove the Minikube Deployment.

Kubernetes deployment design

Kustomize manifests target Minikube and expose a LoadBalancer Service

The manifests define a namespace, two API replicas, health probes, resource requests and limits, and a LoadBalancer Service. Prometheus and Grafana are included in the same Kustomize application.

Build and rollout workflow

- Build `heart-disease-api:latest` from the repository root and load it into Minikube.
- Apply the manifests with `minikube kubectl -- apply -k k8s/` and wait for rollout status.
- Verify both API pods are `1/1` before testing the LoadBalancer endpoints.

Successful Minikube rollout evidence

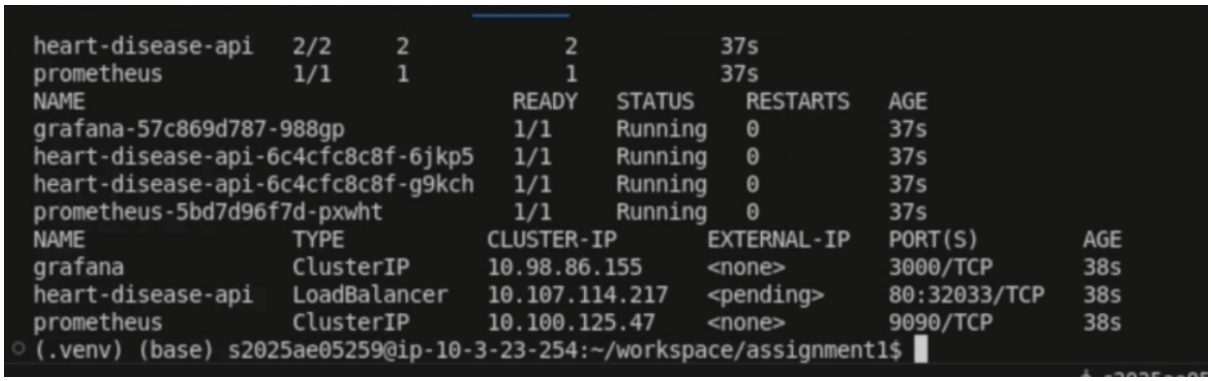


Figure 7. The successful Minikube output shows the API Deployment at 2/2 ready, both API pods `1/1 Running`, and the API LoadBalancer Service.

Successful rollout. The captured output confirms both API replicas and the supporting Prometheus pod are ready. Minikube reports the LoadBalancer external IP as pending; access the service through `minikube service heart-disease-api -n heart-disease --url` or port-forwarding.

Remote VM access

A Minikube service address may be private to the VM. Use `minikube kubectl -- port-forward svc/heart-disease-api -n heart-disease 8000:80`, then test `http://127.0.0.1:8000/health`. Prometheus and Grafana can be forwarded to ports 9090 and 3000 in separate terminals.

The API Deployment uses `imagePullPolicy: IfNotPresent`; the matching image must be built and present in the Minikube runtime.

Monitoring, CI/CD, and final assessment

Operational metrics, automated checks, and submitted evidence

Request middleware logs method, path, status code, and latency. The API exports request, latency, prediction, and model-loaded metrics. Prometheus is configured to scrape the API; Grafana provides the monitoring dashboard.

Grafana monitoring evidence

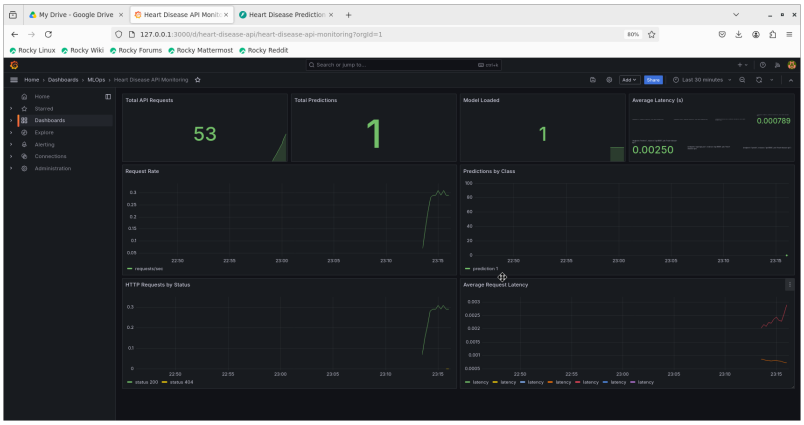


Figure 8. The included dashboard capture shows 53 API requests, 1 prediction, and model_loaded = 1, along with request rate, status, and latency panels.

GitHub Actions and tests

The workflow runs Ruff, model training, Pytest, Docker build, API smoke checks, and artifact upload. Tests cover data processing, both model pipelines, artifact loading, and API routes.

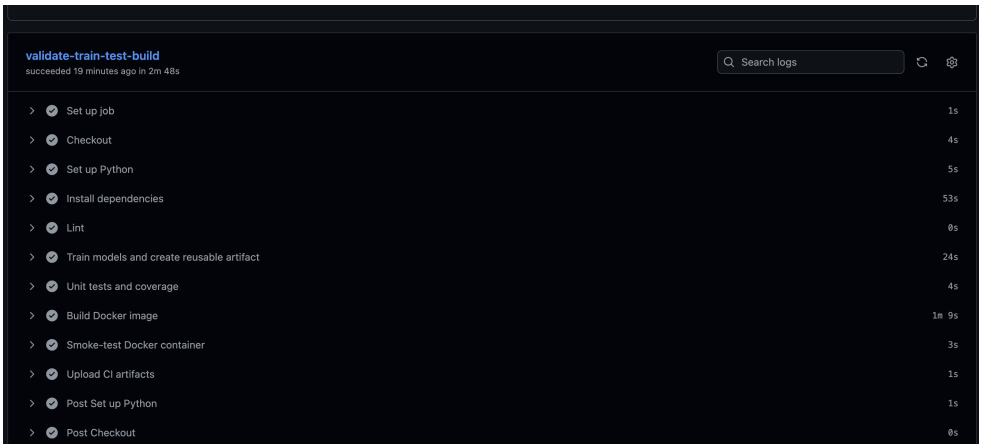


Figure 9. The successful GitHub Actions run shows the `validate-train-test-build` job completing; lint, training, unit tests, Docker image build, container smoke test, and artifact upload all passed.

The companion video follows data preparation and EDA through training, MLflow, API prediction, Minikube deployment, Prometheus/Grafana monitoring, and the submitted repository.

Conclusion

The repository implements data preparation, model selection, MLflow tracking, API inference, Docker, Kubernetes manifests, tests, CI, request logging, and monitoring configuration. The submitted successful CI run and healthy two-replica Minikube rollout provide deployment evidence. Saved holdout results are an assignment benchmark, not clinical validation.

Code repository: https://github.com/NandiniSharma05022002/heart_disease_assignment1
Companion demo: mlops_assignment1_demo_vid.mp4
Dataset: Janosi et al., UCI Heart Disease, DOI 10.24432/C52P4X.